

PRACTICAL 6

Power Rangers

The Sacred Elements of the Faith

the holy
origins

the holy
structures

107 FM Factory Method	the holy behaviors						139 A Adapter	
117 PT Prototype	127 S Singleton					223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade	
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge	

Rayhaan Ahmed (u25047762)

Tashil Haripersad (u25466004)

Kyle Naicker (u25040040)

Haytham Phillips (u25030907)

Oratile Lesedi Rabotapi (u25243269)

TABLE OF CONTENTS

Task 1: Research Workflow Management Systems	3
Task 2: Define the Scenario and System	5
Task 3: UML Activity Diagrams	7
Task 4: Design Patterns	10
Task 5: UML Class Diagram	15
Task 6: Runtime Behaviour Diagrams	16
Task 7: Design Decisions and Revisions	19
Task 8: Github Workflow.....	20

TASK 1: RESEARCH WORKFLOW MANAGEMENT SYSTEMS

Workflow Management Systems Architecture and Design

A workflow automates business processes by routing tasks, information, and documents between participants based on procedural rules [1]. A Workflow Management System (WfMS) is the software defining and executing these processes via an enactment engine [5]. For a system like TIVIDY, the architecture must decouple generic execution mechanics from specific business domains.

Definition vs. Instance

The fundamental design principle is the strict separation of a static *workflow definition* from a dynamic *running instance* [4]. The definition acts as a blueprint, containing the activity graph, role mappings, and rule-based logic [2], [8]. The instance holds runtime state, active payload data, and historical logs [1]. This isolation ensures the core engine remains domain-agnostic, simply traversing the graph without requiring business-specific knowledge.

Task Representation, Routing, and Logic

Work items are represented as discrete nodes. Upon reaching a node, the engine generates an item in a worklist [1]. Routing is governed by gateways that evaluate instance data. To enable dynamic resource allocation, work is assigned to abstract roles or groups rather than hard-coded individuals [3], often linking to statuses (e.g., 'In Progress') for better project visibility [6].

Approvals branch the workflow based on user input, while escalations rely on boundary timers to trigger alternative paths if an SLA is breached [2], [3]. To maximize flexibility, automated routing decisions, thresholds, and role assignments are driven by an external Business Rules Engine (BRE) rather than being hard-coded into the graph [4], [7].

Handling Complexity and Integration

Enterprise workflows manage complexity through nested sub-processes, allowing parent workflows to invoke reusable child definitions [3]. Furthermore, workflows interact with external systems via asynchronous service tasks, utilizing message queues to orchestrate data across third-party applications without blocking the engine's main execution thread [1], [8].

Lifecycle Management, Auditing, and Versioning

Complete operational visibility requires every state transition to be recorded in an append-only audit log, physically separated from active instances to aid compliance and process mining [4], [6]. Dashboards monitor these active worklists to help optimize real-time resource usage [7], [8]. Finally, because processes evolve, systems handle in-flight changes via strict versioning: active instances remain "pinned" to their original blueprint,

while new instances instantiate using the updated definition to ensure structural consistency [4].

References

- [1] D. Hollingsworth, "The Workflow Reference Model," Workflow Management Coalition, Document Number TC00-1003, 1995. [Online]. Available: https://www.researchgate.net/publication/340275372_Realization. [Accessed: Oct. 3, 2026].
- [2] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow Patterns," *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5-51, 2003.
- [3] Object Management Group (OMG), "Business Process Model and Notation (BPMN) Version 2.0," OMG, 2011. [Online]. Available: <https://www.omg.org/spec/BPMN/2.0/>. [Accessed: Oct. 3, 2026].
- [4] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, 2nd ed. Berlin, Germany: Springer, 2018.
- [5] ScienceDirect, "Workflow Management System," ScienceDirect. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/workflow-management-system>. [Accessed: Oct. 4, 2026].
- [6] Atlassian, "What is Workflow Management?," Atlassian. [Online]. Available: <https://www.atlassian.com/agile/project-management/workflow-management>. [Accessed: Oct. 4, 2026].
- [7] SAP, "Workflow Management," SAP. [Online]. Available: <https://www.sap.com/resources/workflow-management>. [Accessed: Oct. 4, 2026].
- [8] Microsoft, "What are Workflow Automation Tools and Software?," Microsoft Power Automate. [Online]. Available: <https://www.microsoft.com/en-za/power-platform/products/power-automate/topics/workflow-automation/workflow-automation-software-and-tools>. [Accessed: Oct. 4, 2026].

TASK 2: DEFINE THE SCENARIO AND SYSTEM

Initial Scope of TIVIDY

TIVIDY is a generic engine that stores workflow definitions, creates independent instances from them and runs them to a final outcome. Its main responsibilities are:

- **Definition management:** build and store immutable, versioned definitions.
- **Instance management:** create, start, cancel and complete instances, each with its own data and history.
- **Execution:** make work available when its dependencies are met, run work sequentially or in parallel (fork and join), and detect completion.
- **Work-item lifecycle:** enforce which operations are valid in which state.
- **Assignment:** choose a participant for each piece of work using a replaceable rule.
- **Approval and escalation:** pass decisions through configurable levels.
- **Events:** react to completions and to external events.
- **Integration:** talk to outside systems through one interface.
- **History:** record actions, support undo for workflow records, and keep an audit trail.
- **Reporting:** produce progress views without adding report code to the work model.

Boundaries: Domain vocabulary such as substation names, areas and voltages lives in data payloads, and roles are plain strings.

Definition versus instance: The definition holds the work tree (stages, items, sub-processes), dependencies, required roles, the assignment strategy, the approval chain and the version number. The instance holds a cloned work tree with live states, assignees, data, the action history and the definition version it was created from.

Expected variation: Process structure, assignment rules, approval chains, added behaviour on tasks (validation, auditing, notification), external integrations, event reactions and reports. The work-item lifecycle is fixed by the engine.

Chosen Scenario: Primary Substation Fault Response and Restoration

Organisation. The City of Tshwane's Energy and Electricity Business Unit (EEBU). Tshwane receives bulk electricity from Eskom at the Njala Infeed Substation. Municipalities buy bulk supply from Eskom and distribute it within their boundaries.

Trigger. On 29 September 2026 an explosion at the Waltloo substation during heavy rain caused a 132 kV trip at Njala and affected at least 11 primary substations. A Njala transformer was found faulty, and the City introduced three-hour load rotations while installing interconnectors to draw supply from other substations. Our group experienced the resulting outages and rotation around Hatfield. TIVIDY models the City's response process, not the electrical network.

Participants and roles. Control room operator, high-voltage (HV) technician, field crew, emergency services, equipment manufacturer (OEM) engineer, EEBU manager, communications officer. Eskom is an external party.

Major pieces of work.

1. Log the incident, triage it and identify the affected primary substations.
2. Make the site safe, in parallel: emergency services secure the site, HV technicians isolate equipment, and communications issues the first public notice.
3. Assess the damage (only after the site is declared safe).
4. Decide whether load rotation is needed. If so, run a rotation sub-process (publish the schedule, switch area blocks every slot, repeat) alongside the repair.
5. Repair, replace, or reroute supply through interconnectors; escalate to the OEM if the equipment cannot be repaired on site.
6. Test and commission.
7. Approve re-energising.
8. Restore supply, end rotation and notify the public.
9. Close the incident and produce a report.

Decisions and routing: Are primary substations affected? Is the site safe? Is rotation needed? Is the fault repairable on site? Did the test pass? Was re-energising approved?

Approvals and dependencies: Re-energising needs approval through a chain (technician lead, then EEBU manager, then executive). Assessment depends on site safety. Re-energising depends on a passed test. Rotation ends only after restoration.

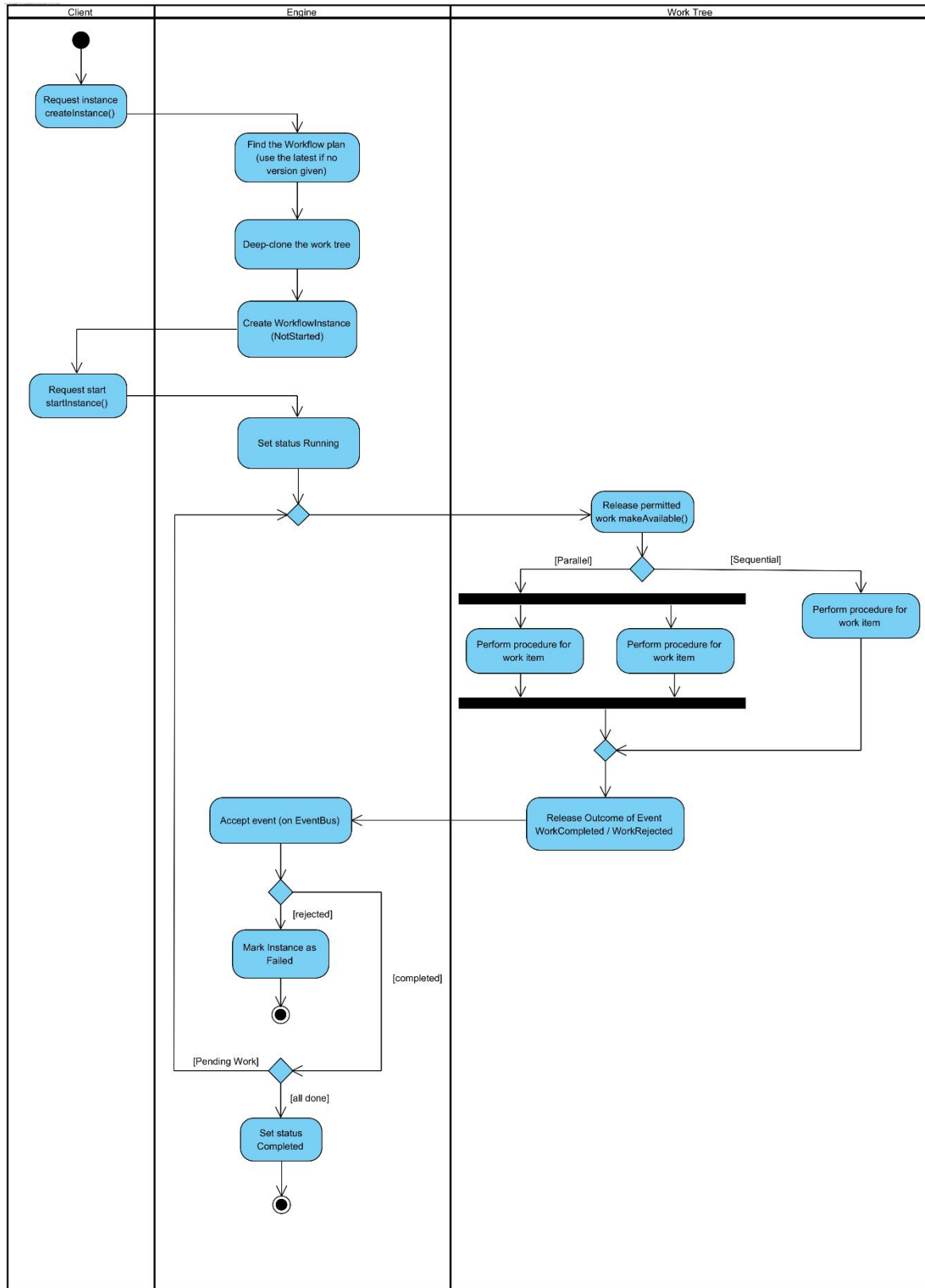
Failures and exceptions: An attempt to re-energise the faulty transformer was unsuccessful, and persistent rain delayed repair work. We also model: no qualified HV technician available, a failed test, an approval that goes unanswered (escalation), and a failed public notification.

External systems: Eskom (bulk supply notifications), emergency services, the equipment OEM and public communication channels such as the City's published updates. These are reached through adapters.

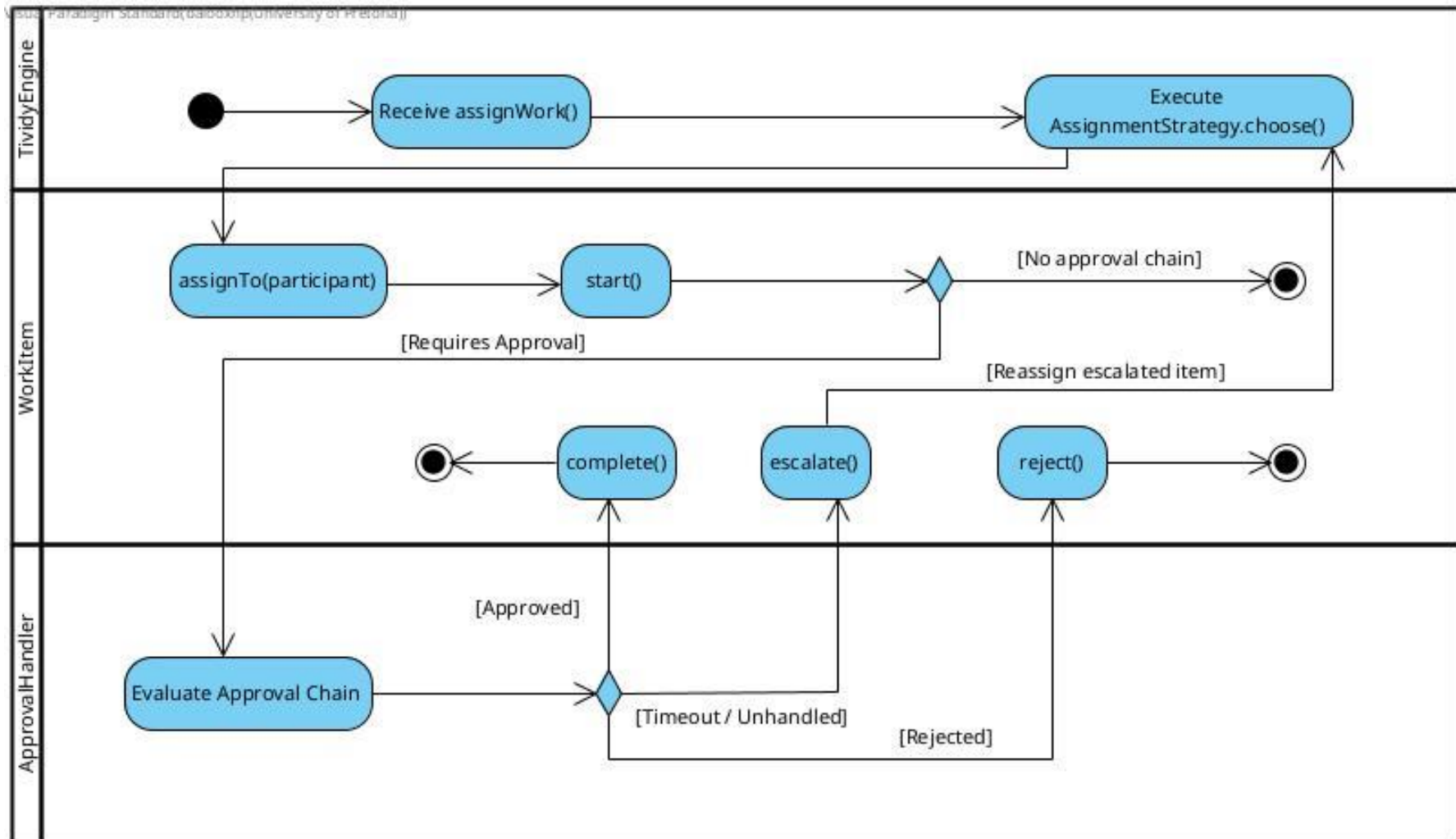
Expected outcome: Supply restored to all affected areas, rotation ended, and a complete audit trail and report, or a documented residual risk with a permanent fix scheduled.

TASK 3: UML ACTIVITY DIAGRAMS

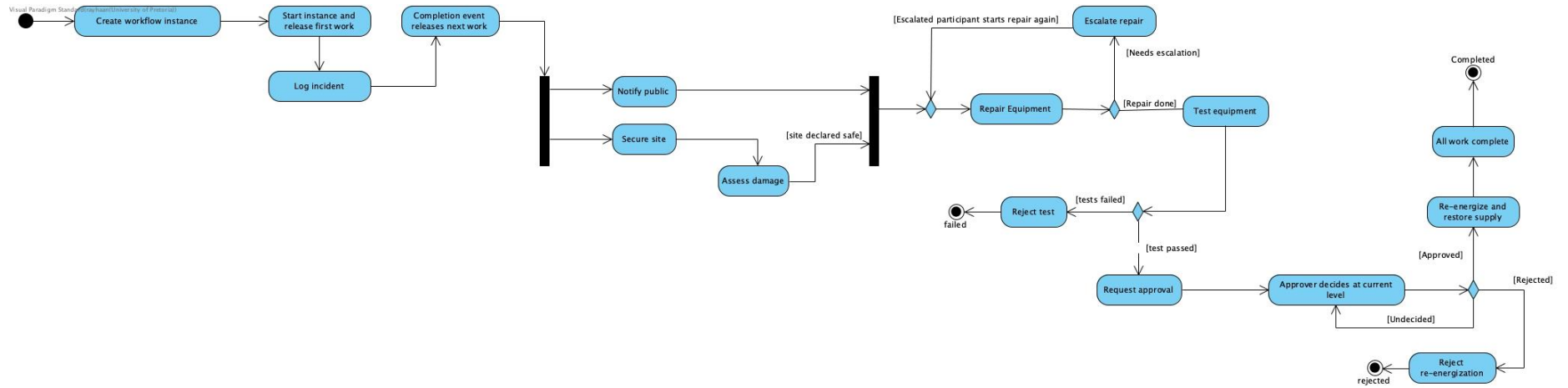
Activity Diagram 1



Activity Diagram 2



Activity Diagram 3



TASK 4: DESIGN PATTERNS

1. Prototype

- **Problem:** the brief asks us to separate a definition from a running process, and says a change to one running instance must not change every other instance. Each fault incident needs its own private copy of the response process.
- **Participants:** Prototype = WorkComponent (clone). ConcretePrototypes = WorkItem, WorkGroup and the decorators. Client = WorkflowDefinition::instantiate.
- **Collaboration:** instantiate asks the root component to clone itself, and each group clones its children. Clones start in the Created state with no assignee.
- **Varies:** the content of each process and the number of instances created from it.
- **Why:** re-running the builder per incident would couple execution to construction and lose the versioned snapshot. Cloning guarantees isolation and keeps definitions immutable.

2. Builder

- **Problem:** 'workflow construction and execution' are separate concerns, and processes contain nested stages and sub-processes. A fault-response definition has groups inside groups.
- **Participants:** Builder/ConcreteBuilder = WorkflowBuilder. Product = WorkflowDefinition. Director = scenario code.
- **Collaboration:** the director calls beginGroup, addItem, dependsOn, requireApproval and endGroup. The builder keeps a stack of open groups and returns a finished definition from build.
- **Varies:** which processes are defined, without changing the engine.
- **Why:** hand-assembling nested trees is error-prone, and the builder keeps the construction rules (every item belongs to a group, dependencies point at known items) in one place.

3. Composite

- **Problem:** work is 'rarely flat': stages, items and sub-processes, some sequential and some parallel. The engine should treat 'make site safe' and 'isolate equipment' the same way.
- **Participants:** Component = WorkComponent. Leaf = WorkItem. Composite = WorkGroup (Sequential or Parallel).
- **Collaboration:** calls such as cancel or advance on a group are forwarded to its children. A parallel group releases all eligible children (fork) and completes when all finish (join).
- **Varies:** the nesting depth and whether a stage runs in sequence or in parallel.
- **Why:** the rest of the system does not need to know how work is stored or nested, which the brief explicitly requires.

4. Iterator

- **Problem:** work must be navigated 'without requiring the rest of the system to know their internal storage'. The engine and reports must visit every item to find what is ready.
- **Participants:** Iterator = WorkIterator. ConcreteIterator = DepthFirstIterator. Aggregate = WorkComponent (createIterator).
- **Collaboration:** a client calls first, isDone, next and current while the iterator walks the tree depth first.
- **Varies:** traversal order (for example breadth first or only available items) and the internal storage.
- **Why:** without it, every caller would recurse through children itself and break if storage changed.

5. Visitor

- **Problem:** reports must be produced 'without placing every possible reporting operation inside the core model'. Managers need progress and status counts for an incident.
- **Participants:** Visitor = WorkVisitor. ConcreteVisitor = ProgressReport. Elements = WorkItem and WorkGroup (accept).
- **Collaboration:** report asks the root to accept the visitor. Groups visit themselves and then their children, and decorators forward accept to the item they wrap.
- **Varies:** the set of reports (progress, overdue work, workload per participant).
- **Why:** a new report is a new visitor class and the work classes are untouched.

6. State

- **Problem:** a work item can be 'created, made available, assigned, started, completed, rejected, cancelled or escalated', and operations valid at one point are invalid at another. Completing an item that was never started must be refused.
- **Participants:** Context = WorkItem. State = WorkItemState. ConcreteStates = CreatedState, AvailableState, AssignedState, InProgressState, EscalatedState, CompletedState, RejectedState, CancelledState.
- **Collaboration:** WorkItem forwards each operation to its current state. A legal transition calls setState and publishes an event. Illegal operations are refused by the base class.
- **Varies:** the allowed transitions per state.
- **Why:** the brief bans large if/else chains, and a status switch in every operation would become that.

7. Strategy

- **Problem:** 'one organisation may assign the next task based on role, another on workload, and another on a rule calculated from workflow data'. In our scenario, HV work must go to HV-qualified technicians.

- **Participants:** Strategy = AssignmentStrategy. ConcreteStrategies = RoleAssignment, LeastLoadedAssignment, RuleAssignment. Context = WorkflowInstance (holds the strategy from its definition).
- **Collaboration:** when work becomes available the engine passes the item and the participant pool to choose, which returns a participant (or none, which can trigger escalation).
- **Varies:** the assignment rule per process or organisation.
- **Why:** the brief says such rules must not force rewriting the work-item classes.

8. Chain of Responsibility

- **Problem:** 'approval may involve one person, several levels, or escalation when nobody responds'. Re-energising a substation is approved by the technician lead, then the EEBU manager, then an executive.
- **Participants:** Handler = ApprovalHandler. ConcreteHandlers = LevelApprover, FallbackApprover. Client = TividyEngine::requestApproval.
- **Collaboration:** handle calls the template step decide. If the request is still undecided (for example after a timeout), it is passed to the next handler. The fallback ends the chain.
- **Varies:** the number of levels, their roles and timeouts.
- **Why:** escalation becomes configuration, so one, two or three levels need no engine change. Template Method is used inside handle and decide.

9. Decorator

- **Problem:** a task 'may acquire additional behaviour such as validation, priority handling, auditing, notifications or security without becoming a completely different kind of task'. For example, 'approve re-energising' needs validation and notification, while 'secure site' does not.
- **Participants:** Component = WorkComponent. Decorator = WorkItemDecorator. ConcreteDecorators = ValidatedWork, AuditedWork, NotifyingWork.
- **Collaboration:** a decorator wraps any component, runs its extra step before or after forwarding, and can be stacked. clone re-wraps the cloned inner item so decoration survives instantiation.
- **Varies:** which behaviours each task carries.
- **Why:** subclassing every combination of behaviours would multiply classes.

10. Observer

- **Problem:** 'completing one activity may make another available', an external event may trigger work, and 'several interested parts of the system may need to react to the same event'.
- **Participants:** Subject = EventBus. Observer = EventObserver. ConcreteObservers = WorkflowInstance and ConnectorObserver.
- **Collaboration:** work items publish events such as WorkCompleted. The instance advances the tree, and a connector observer can forward the event to an external

system. External events such as a rain delay or an Eskom notice enter through `publishExternalEvent`.

- **Varies:** who listens, and which events exist.
- **Why:** new listeners (a dashboard, a notifier) are added by subscribing, with no change to the work classes.

11. Command

- **Problem:** the system must 'retain actions and state changes' and 'support undo or rollback where sensible'. Every assignment, start, completion and rejection must be recorded.
- **Participants:** Command = Action. ConcreteCommands = AssignAction, StartAction, CompleteAction, RejectAction, CancelAction (via WorkAction). Invoker = ActionHistory. Receiver = WorkComponent.
- **Collaboration:** the engine creates an action and ActionHistory runs it, logs a description and stacks it for undo. WorkAction::execute is a template method that takes a snapshot, then calls the subclass step perform.
- **Varies:** the set of operations and what is logged.
- **Why:** the history is the audit trail, and wrongly assigned work can be undone in the workflow record.

12. Memento

- **Problem:** undo needs to restore earlier state without exposing a work item's internals.
- **Participants:** Originator = WorkItem (save, restore). Memento = WorkMemento. Caretaker = WorkAction.
- **Collaboration:** before performing, the action stores a memento of the item's status, assignee and data. Undo hands it back to restore.
- **Varies:** what state is captured.
- **Why:** only the work item can read the snapshot, so encapsulation is kept. Undo applies to workflow records only. Physical switching in the scenario needs compensating work instead.

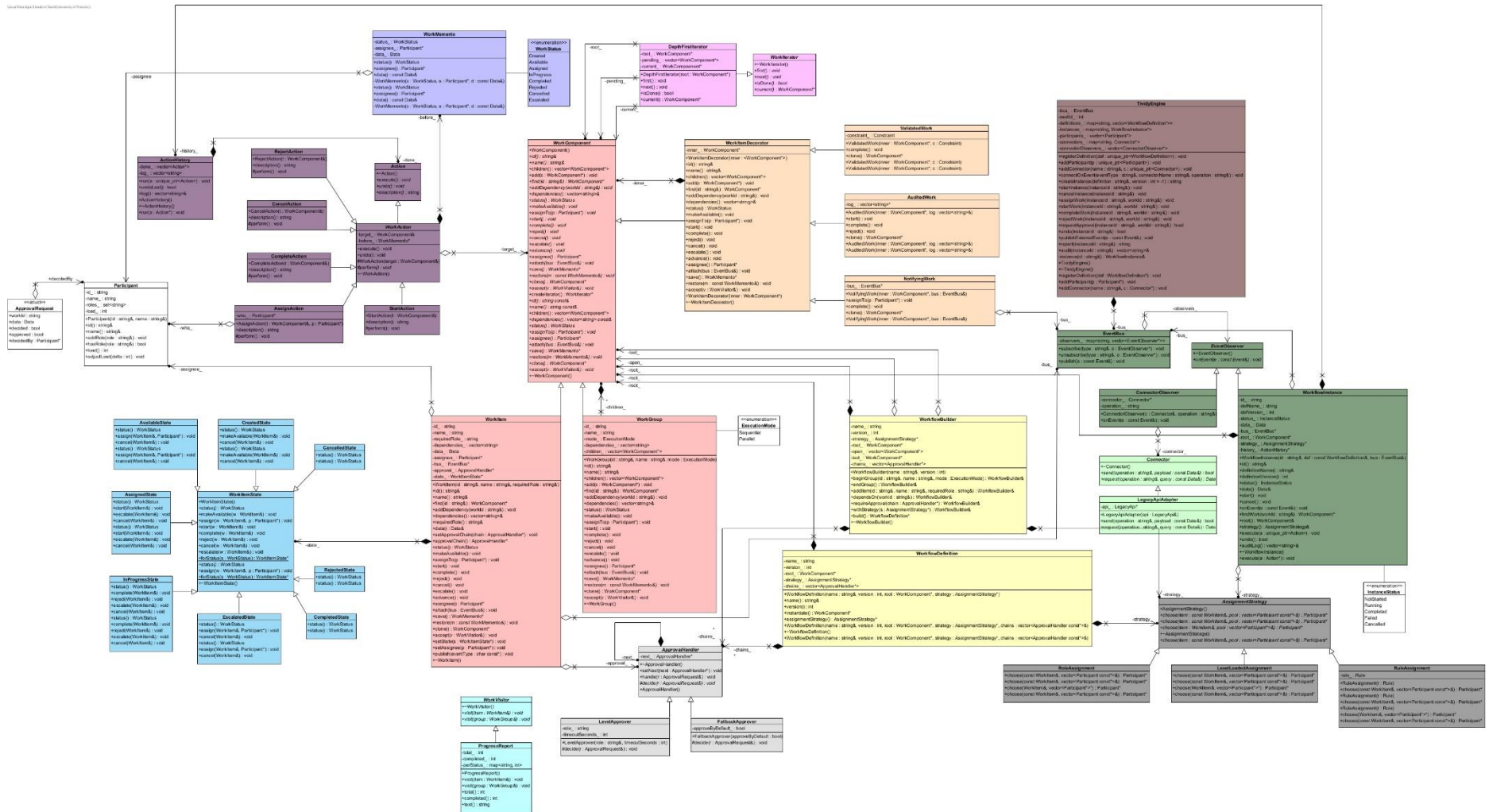
13. Adapter

- **Problem:** TIVIDY 'may also need to communicate with external systems whose interfaces were not designed for it'. In our scenario these include Eskom notices, public notification channels and the OEM.
- **Participants:** Target = Connector. Adaptee = the external system's own API (forward-declared LegacyApi). Adapter = LegacyApiAdapter (one concrete adapter per external system).
- **Collaboration:** TIVIDY calls send or request on the Connector and the adapter translates to the external API and back.
- **Varies:** the external systems and their interfaces.
- **Why:** the engine depends on one interface, so replacing a system touches one adapter.

14. Facade

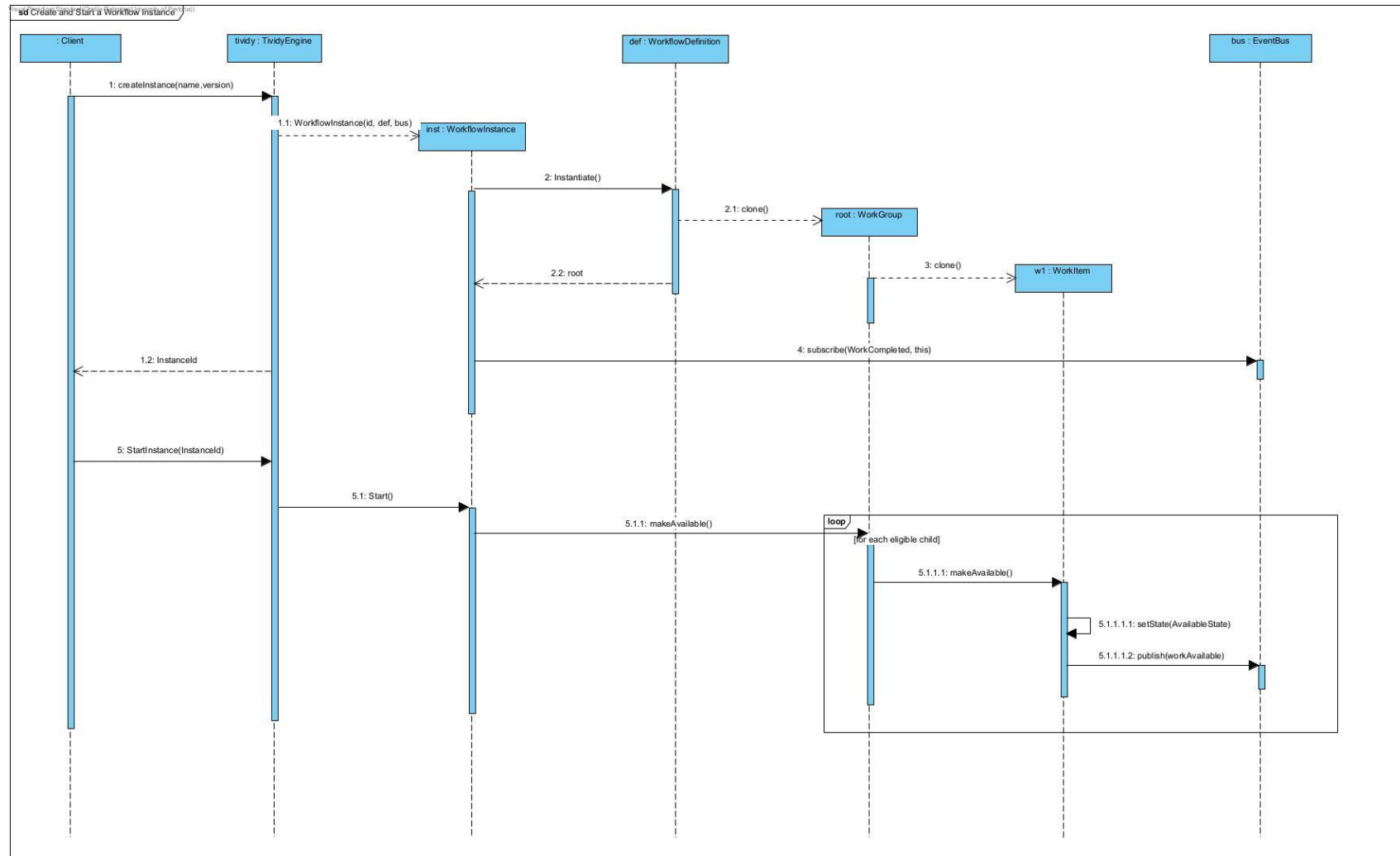
- **Problem:** the engine contains many cooperating parts, and scenario code and users need one simple way in.
- **Participants:** Facade = TividyEngine. Subsystems = definitions, instances, participants, connectors, EventBus, ActionHistory.
- **Collaboration:** a call such as completeWork finds the instance and work item, builds the action, runs it through the history and lets events propagate.
- **Varies:** the internals behind the interface.
- **Why:** callers stay independent of the subsystem classes.

TASK 5: UML CLASS DIAGRAM

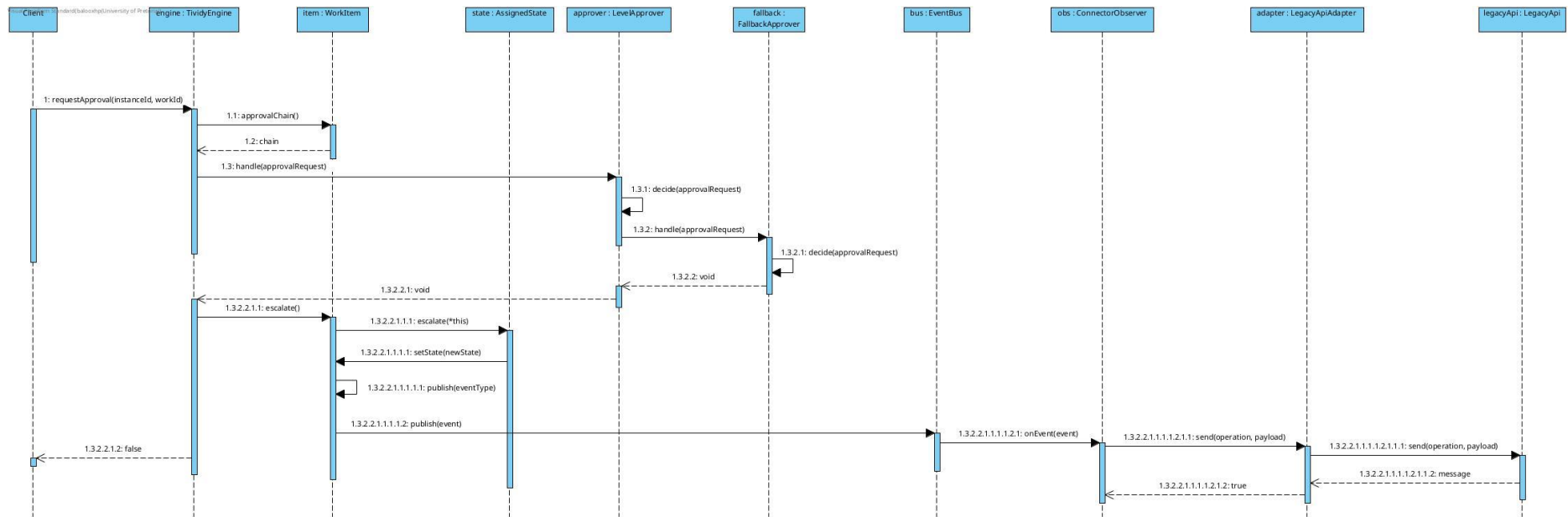


TASK 6: RUNTIME BEHAVIOUR DIAGRAMS

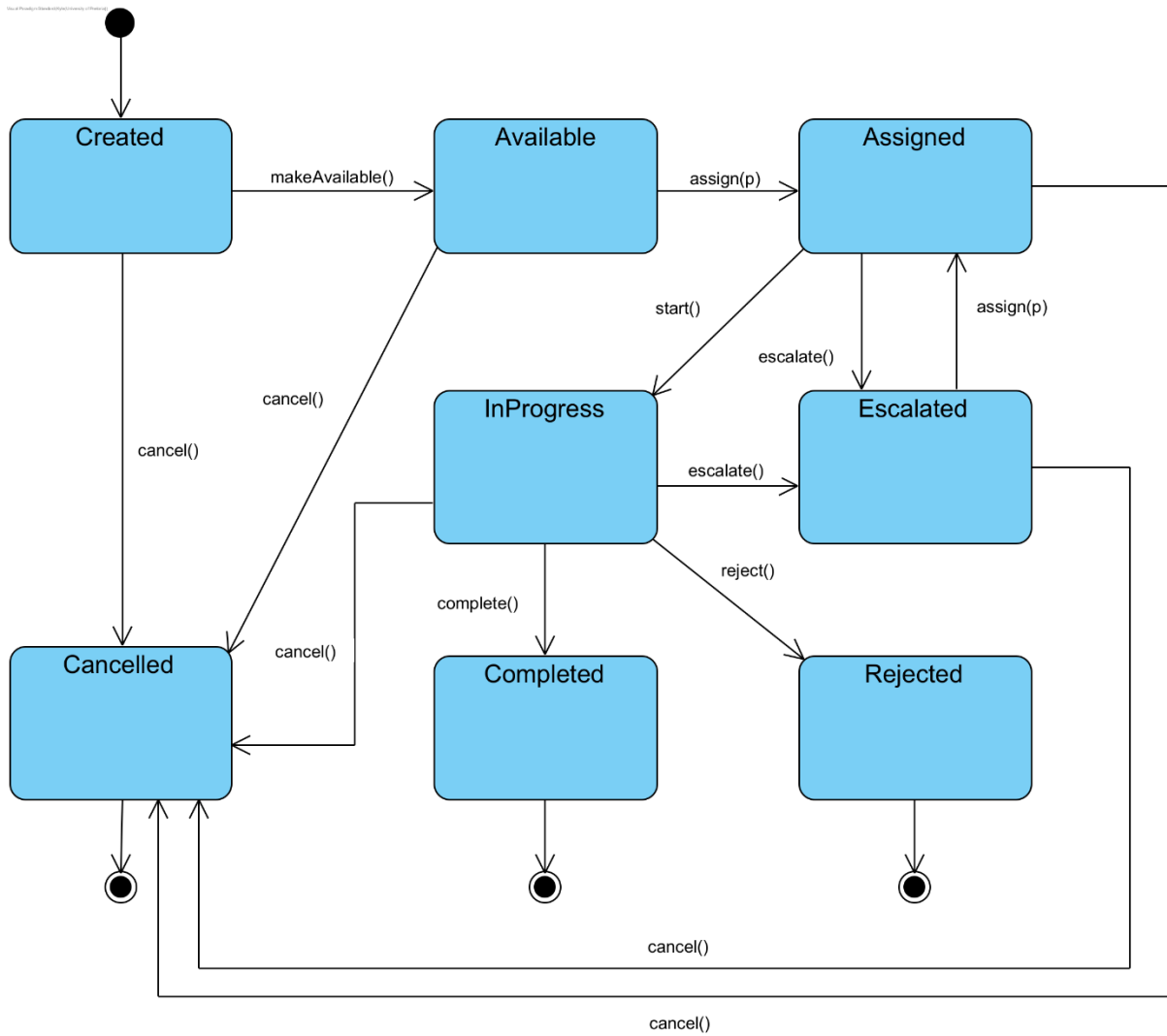
Sequence Diagram 1



Sequence Diagram 2



State Diagram



TASK 7: DESIGN DECISIONS AND REVISIONS

Initial design decisions

- **Lifecycle operations live on WorkComponent:** Question: how can Decorator wrap tasks? Decision: start, complete and similar operations are on the common interface, not only on WorkItem, so a decorator is interchangeable with a task. Groups implement them by delegating to children. Diagrams: class diagram.
- **Definitions are immutable and versioned:** Decision: running instances keep the version they started with, as in [3]. Diagrams: class diagram, sequence diagram for creating an instance.
- **Undo covers leaf work items only:** Groups derive their status from their children, so a group cannot be snapshotted. Diagrams: class diagram.
- **Roles and event types are strings:** This keeps the engine generic and lets scenarios define vocabulary. Trade-off: no compile-time checking.

Revisions

- **Scope of the model:** Problem: should TIVIDY model the electrical network? Original: unclear. Change: it models the response process only, and network details travel as data. Why: the brief says the scenario must not determine the architecture.

TASK 8: GITHUB WORKFLOW

GitHub Repository Link

<https://github.com/HaythamP/TIVIDY.git>

Team Contribution Statement

All members contributed to the research and design.

- **Rayhaan:** Activity Diagram 3
- **Tashil:** compilation of the PDF
- **Kyle:** Activity Diagram 1, State Diagram
- **Haytham:** Activity Diagram 2, Sequence Diagram 2
- **Oratile:** Sequence Diagram 1